

Program – 2

Develop a Multi-Stage Dockerfile for Container Orchestration

Program Structure

```
program2/  
----->node_modules  
----->package.json  
----->package-lock.json  
----->build|  
--->src  
----->index.js  
----->dist  
----->index.js  
----->Dockerfile
```

Step 1: Create a project folder

```
mkdir program2  
cd program2
```

Step 2 : initialize npm – it will create package.json

```
npm init -y
```

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~$ mkdir program2  
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~$ cd program2  
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ nano Dockerfile  
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ mkdir src  
nano src/index.js  
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ nano package.json  
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ npm init -y  
Wrote to /home/1RV24MC073_pavithra/program2/package.json:
```

```
{  
  "name": "program2",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "keywords": [],  
  "author": "",  
  "license": "ISC"  
}
```

Step 3 : create a src folder and create index.js file inside this folder

```
mkdir src  
cd src
```

nano index.js

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/program2/src
GNU nano 6.2 index.js
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from multi-stage Docker!');
});

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 4 : create a docker file
nano Dockerfile

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/program2
GNU nano 6.2 Dockerfile
# Stage 1: Build Stage
FROM node:20-alpine AS builder

# Set working directory
WORKDIR /app

# Copy package files and install dependencies
COPY package.json package-lock.json ./
RUN npm install

# Copy application source code
COPY . .

# Build the application
RUN npm run build

# Stage 2: Production Stage
FROM node:20-alpine

# Set working directory
WORKDIR /app

# Copy only necessary files from build stage
COPY --from=builder /app/package.json ./
COPY --from=builder /app/package-lock.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules

# Expose the application port
EXPOSE 3000

# Start the application
CMD ["node", "dist/index.js"]
```

Step 5 : build docker image

docker build -t program2 .

run the container

docker run -p 3000:3000 program2

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/program2
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ nano package.json
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ npm run build

> program-2@1.0.0 build
> mkdir -p dist && cp -r src/* dist/

1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ docker build -t program2 .
[+] Building 15.2s (16/16) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 792B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  2.1s
=> [auth] library/node:pull token for registry-1.docker.io        0.0s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:658d0f63e501824d6c23e06d4bb95c71e7d704537c9d9272f488 0.0s
=> [internal] load build context                                    0.0s
=> => transferring context: 811B                                      0.0s
=> CACHED [builder 2/6] WORKDIR /app                               0.0s
=> [builder 3/6] COPY package.json package-lock.json ./           0.1s
=> [builder 4/6] RUN npm install                                   11.3s
=> [builder 5/6] COPY . .                                          0.1s
=> [builder 6/6] RUN npm run build                                 0.7s
=> [stage-1 3/6] COPY --from=builder /app/package.json ./         0.1s
=> [stage-1 4/6] COPY --from=builder /app/package-lock.json ./    0.1s
=> [stage-1 5/6] COPY --from=builder /app/dist ./dist             0.1s
=> [stage-1 6/6] COPY --from=builder /app/node_modules ./node_modules 0.2s
=> exporting to image                                              0.2s
=> => exporting layers                                              0.1s
=> => writing image sha256:40a89f497c951f3e77c2cd51956ade09875c412797f55af53a673d14d76dc3f7 0.0s
=> => naming to docker.io/library/program2                        0.0s
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/program2$ docker run -it -p 3000:3000 program2
Server running on port 3000
```

Step 7 : verify the application

open the browser and go to <http://localhost:3000>



